

A top-down view of a workspace featuring a laptop, a separate keyboard, a spiral-bound notebook, a pair of glasses, a pen, and a mouse. The word "LINQ" is centered in a large, bold, black font. Two teal-colored geometric shapes, resembling chevrons, are positioned in the top-left and bottom-right corners.

LINQ

¿QUÉ ES LINQ?

LINQ (Language Integrated Query) es un conjunto de características introducidas en el lenguaje C# que permite realizar consultas sobre colecciones de datos de una manera similar a SQL.

SUS VENTAJAS

Legibilidad: Código más limpio y fácil de entender.

Productividad: Menos código y más funcionalidades.

Consistencia: Una sola forma de consultar datos de diferentes fuentes (arrays, listas, bases de datos).

SINTAXIS BÁSICA DE LINQ

```
public List<Enemy> enemies;

void Start()
{
    List<Enemy> weakEnemies = enemies.Where(enemy => enemy.health < 50).ToList();

    foreach (var enemy in weakEnemies)
    {
        Debug.Log("Enemy: " + enemy.name + " - Health: " + enemy.health);
    }
}
```

El método **Where** en LINQ (Language Integrated Query) se utiliza para filtrar elementos de una colección basada en una condición específica. En el ejemplo que has proporcionado, estás utilizando LINQ para filtrar una lista de enemigos (enemies) y obtener solo aquellos enemigos cuya salud (health) es menor a 50

SINTAXIS BÁSICA DE LINQ

```
List<Enemy> enemyNames = (List<Enemy>)enemies.OrderBy(c => c.enemyName.ToList());

foreach (var enemy in enemyNames)
{
    Debug.Log("Collectible: " + enemy.name + " - Health: " + enemy.health);
}
```

- **enemies:** Es una lista que contiene objetos de tipo Enemy.

- **OrderBy(c => c.enemyName):** OrderBy es un método de LINQ que se utiliza para ordenar una colección en función de una clave. En este caso, estamos ordenando la colección enemies por el atributo enemyName de cada objeto Enemy.

- **c => c.enemyName** es una expresión lambda que especifica la clave de ordenación (el nombre del enemigo).

- **.ToList():** Convierte el resultado ordenado (que es un IEnumerable<Enemy>) en una lista (List<Enemy>).

SINTAXIS BÁSICA DE LINQ

Utilizar LINQ para proyectar una colección de objetos en una nueva forma (en este caso, una lista de nombres) y luego recorrer esa nueva colección para realizar acciones en cada elemento.

```
var enemyNames = enemies.Select(e => e.name).ToList();

foreach (var name in enemyNames)
{
    Debug.Log("Enemy Name: " + name);
}
```

- **LINQ:** Una característica de C# que permite realizar consultas y operaciones sobre colecciones de datos de manera declarativa.
- **Expresión Lambda:** Una forma concisa de definir funciones anónimas. En este caso, `e => e.name` es una expresión lambda que selecciona el nombre de cada enemigo.
- **Método de extensión:** `Select` es un método de extensión que extiende la funcionalidad de las colecciones que implementan `IEnumerable<T>`.

SINTAXIS BÁSICA DE LINQ

El objetivo de este código es encontrar el primer enemigo en una lista de enemigos (enemies) que tenga una salud superior a 100, y si se encuentra tal enemigo, imprimir su nombre en la consola.

```
var strongEnemy = enemies.FirstOrDefault(e => e.health > 100);  
  
if (strongEnemy != null)  
{  
    Debug.Log("First strong enemy: " + strongEnemy.name);  
}
```

- **FirstOrDefault(e => e.health > 100):** **FirstOrDefault** es un método de LINQ que se utiliza para encontrar el primer elemento en una colección que cumple con una condición especificada.
- **e => e.health > 100** es una expresión lambda que define la condición. e es cada elemento de la colección **enemies**, y e.health > 100 es la condición que debe cumplirse (la salud del enemigo debe ser mayor a 100).
- Si se encuentra un enemigo que cumple con la condición, FirstOrDefault devuelve ese enemigo. Si no se encuentra ningún enemigo que cumpla con la condición, FirstOrDefault devuelve null.

SINTAXIS BÁSICA DE LINQ

Contar Elementos que Cumplen una Condición

```
var weakEnemiesCount = enemies.Count(e => e.health < 50);  
Debug.Log("Number of weak enemies: " + weakEnemiesCount);
```

- **enemies:** Es una lista que contiene objetos de tipo Enemy.
- **Count(e => e.health < 50):** **Count** es un método de LINQ que se utiliza para contar el número de elementos en una colección que cumplen con una condición especificada.
- **e => e.health < 50** es una expresión lambda que define la condición. e es cada elemento de la colección enemies, y e.health < 50 es la condición que debe cumplirse (la salud del enemigo debe ser menor a 50).
- El resultado de Count es un entero (int) que representa el número de enemigos que cumplen con la condición.

SINTAXIS BÁSICA DE LINQ

El objetivo de este código es combinar los nombres de dos listas diferentes (una lista de enemigos y una lista de ítems) y luego imprimir todos los nombres combinados en la consola.

```
var enemyNames = enemies.Select(e => e.name);  
var itemsNames = items.Select(c => c.name);  
  
var allNames = enemyNames.Concat(itemsNames).ToList();  
  
foreach (var name in allNames)  
{  
    Debug.Log("Name: " + name);  
}
```

- **enemyNames.Concat(itemsNames): Concat** es un método de LINQ que se utiliza para concatenar dos secuencias. En este caso, concatenamos enemyNames con itemsNames.

- **.ToList():** Convierte el resultado de Concat (que es un IEnumerable<string>) en una lista (List<string>).

- El resultado es una lista llamada allNames que contiene todos los nombres de enemigos y todos los nombres de ítems combinados.

RESUMEN

Gestión de Inventarios: Filtrar, ordenar y agrupar objetos en el inventario.

AI y Enemigos: Seleccionar enemigos dentro de un rango de salud específico o de una cierta clase.

Datos de Jugador: Consultar y agrupar estadísticas de jugadores.

Manejo de Escenas: Encontrar objetos específicos en la escena (ej. objetos con ciertos componentes).

CONCLUSIÓN

LINQ es una herramienta poderosa para manipular colecciones de datos en Unity, haciendo el código más limpio y eficiente.

Recursos Adicionales

Documentación de LINQ_(Microsoft).